

Predicting Batter–Pitcher Home Run Probabilities Using Statcast Metrics and Lightweight Machine Learning

Jeffrey Chen, Joey DiSalvo, Aidan Feyerherm, Grayson Gill
COSC445
University of Tennessee Knoxville

Abstract—This study investigates whether Statcast pitch-level data can be used to estimate the probability of a specific batter hitting a home run against a specific pitcher. Using a Kaggle-derived MLB Statcast dataset, we evaluate the predictive value of pitch characteristics, game context, and batter/pitcher identifiers. A logistic regression model is applied as a lightweight binary classifier to estimate home run likelihood. Results indicate limited but measurable predictive power, with pitch velocity, pitcher handedness, and pitch location contributing most strongly.

Index Terms—Machine Learning, Baseball Analytics, Statcast, Logistic Regression, Sports Modeling

I. INTRODUCTION

Home runs are among the most unpredictable outcomes in baseball. This project examines whether pitch-level Statcast analytics combined with machine learning provide meaningful predictive capability for batter–pitcher home run events. We develop a modeling pipeline using pitch attributes, contextual metadata, and player identifiers to estimate the probability of a home run occurring.

II. DATA DESCRIPTION

A. Data Source

The dataset originates from Major League Baseball Statcast metrics obtained via Kaggle. It includes pitch-level measurements such as velocity, spin rate, and pitch location, as well as categorical attributes including pitch type, batter ID, pitcher ID, and in-game context (inning, balls, strikes, etc.).

B. Data Obtained and Processed

- Downloaded directly from Kaggle
- Imported using Python
- Removed missing values
- Filtered to retain only pitches resulting in contact
- Standardized numeric variables (velocity, spin, location)

C. Data Validation

- Ensured consistency of Statcast metric columns
- Verified alignment of target variables with pitch outcomes
- Confirmed correctness of pitch type, velocity, and location values
- Validated batter and pitcher identifiers

III. METHODOLOGY

A. Machine Learning Pipeline

A supervised learning pipeline was implemented to predict binary outcomes (home run vs. non-home run). Data preprocessing included one-hot encoding of categorical features and standardization of continuous features.

B. Features Used

- Pitch velocity
- Pitch type
- Horizontal and vertical location
- Count (balls and strikes)
- Batter ID
- Pitcher ID

C. Model Selection

Logistic regression was selected due to its interpretability and suitability for binary classification. Its simplicity is advantageous when modeling complex and noisy baseball performance data.

IV. RESULTS

A. Model Achievements

- Developed a pipeline that ingests pitch data and outputs home run likelihood for a given batter–pitcher matchup.
- Demonstrated that certain pitch-level and contextual variables modestly influence home run probability.

B. Key Takeaways

- Pitcher handedness affects home run odds.
- Pitch velocity influences home run likelihood.
- Logistic regression provides interpretable and stable predictions.
- Unmodeled environmental and contextual factors (ballpark, weather, randomness) limit predictive power.

V. CHALLENGES AND LIMITATIONS

- Difficulty retrieving and validating batter and pitcher IDs.
- Sparsity of positive samples due to home run rarity.
- Weak correlations across many pitch-level variables.
- Feature leakage concerns with advanced Statcast metrics such as exit velocity and launch angle.
- Dataset lacked environmental features such as ballpark dimensions, wall heights, wind, and temperature.

VI. FUTURE WORK

- Expand batter and pitcher identifiers and historical datasets.
- Construct matchup-specific home run probability tables.
- Incorporate improved contextual features (ballpark dimensions, weather, inning, time of day).
- Explore more advanced ensemble models such as Random Forests and XGBoost.
- Create additional visualization tools including strike-zone heatmaps and pitch risk diagrams.

VII. ORGANIZATIONAL STRUCTURE

A project timeline and responsibility chart were developed to assign tasks and track contributions across team members.

- **Grayson Gill: Data Acquisition and Pre-processing** Grayson led all data sourcing and cleaning efforts. This included setting up Python scripts to download Statcast datasets, Odds API feeds, and contextual ballpark data. He merged hitter, pitcher, and stadium tables into a unified format, ensured time consistency (rolling averages by date), and handled missing values. He was also responsible for reproducibility, so the pipeline refreshed daily with minimal manual intervention.
- **Joey DiSalvo: Feature Engineering and Exploratory Analysis** Joey transformed the cleaned data into predictive inputs. This included constructing rolling averages for hitter performance, encoding pitcher HR rates, normalizing launch angle and exit velocity data, and incorporating betting line probabilities as features. He also designed visualizations (heat maps, trend plots, matchup summaries) to identify predictive patterns and validate assumptions.
- **Aidan Feyerherm: Model Development** Aidan focused on building and refining predictive models. They began with interpretable baselines such as logistic regression and random forests, then expanded into more advanced methods such as XGBoost and stacking ensemble models. They also experimented with incorporating contextual “signals” from the OpenAI API to capture higher-level reasoning that complemented numerical data. Hyperparameter tuning, cross-validation, and robustness testing also fell under his responsibility.
- **Jeffrey Chen: Evaluation and System Demonstration** Jeffrey evaluated model performance using precision, recall, and the Area Under the Curve (AUC) for classification accuracy. He also designed the system demonstration, implementing a FastAPI backend to serve predictions and a React-based frontend for user interaction. The demo allowed users to either (a) select a player vs. pitcher matchup and see a predicted HR probability, or (b) input a date to automatically generate the system’s most confident daily home run picks.

Timeline

- **Weeks 1–2: Data Acquisition and Pipeline Setup** We gathered data from Baseball Savant, Statcast, and Odds APIs. We built automated Python scripts for downloading and cleaning the datasets. We merged hitter, pitcher, and ballpark datasets into a single structured format. *Deliverable:* a reproducible and regularly refreshable dataset pipeline.
- **Weeks 3–4: Feature Engineering and Exploratory Analysis** We engineered predictive features such as rolling exit velocity averages, pitcher HR rates by pitch type, and ballpark-adjusted HR factors. We conducted exploratory analysis with plots and statistical summaries to confirm patterns and validate data integrity. *Deliverable:* a feature matrix ready for modeling plus an EDA report highlighting key trends.
- **Weeks 5–6: Model Development and Tuning** We trained baseline models (logistic regression, random forest) to establish benchmarks. We then advanced to ensemble methods such as XGBoost and stacking to improve accuracy. We integrated OpenAI API contextual features, performed hyperparameter tuning, and validated results with cross-validation. *Deliverable:* multiple trained models with documented performance.
- **Week 7: Model Evaluation and Ranking** We compared models using precision, recall, and Area Under the Curve (AUC). We assessed interpretability through feature importance analysis. We also developed a ranking system to highlight the most confident HR predictions for each game. *Deliverable:* evaluation report comparing models, including both accuracy metrics and feature weights.
- **Week 8: Demo Interface and Final Integration** We built the FastAPI backend and React frontend for user interaction. We implemented two modes: matchup-based predictions and full date-based daily picks. We integrated the models into the backend, ensured the pipeline ran smoothly, and finalized the project report. *Deliverable:* fully functional interactive demo and written report.
- **Week 9: Real-Time and Extended Outcomes** We expanded the system to pull live pitch-level data (in-game Statcast feeds) and attempted real-time HR prediction updates. We also explored extending beyond HR outcomes to predict other events such as strikeouts, walks, or doubles. *Deliverable:* a prototype extension showing proof-of-concept for live updating or additional outcome modeling.

VIII. CONCLUSION

This project demonstrates that while predicting home runs remains inherently difficult, lightweight machine learning models applied to Statcast pitch data can provide limited but meaningful insight. Expanding contextual data and applying more robust models may yield improved predictive performance in future work.

REFERENCES

- [1] Major League Baseball, “Statcast Search,” *Baseball Savant*. [Online]. Available: https://baseballsavant.mlb.com/statcast_search [Accessed: Sep. 23, 2025].
- [2] SportsDataIO, “Odds API Documentation,” *SportsDataIO*. [Online]. Available: <https://sportsdata.io/developers/api-documentation/odds> [Accessed: Sep. 23, 2025].
- [3] Google Cloud, “Statcast Data with BigQuery,” *Google Cloud Public Datasets*. [Online]. Available: <https://cloud.google.com/bigquery/public-data/statcast> [Accessed: Sep. 23, 2025].
- [4] OpenAI, “API Reference,” *OpenAI Platform Documentation*. [Online]. Available: <https://platform.openai.com/docs/api-reference> [Accessed: Sep. 23, 2025].
- [5] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [6] FastAPI, “FastAPI Documentation,” *FastAPI*. [Online]. Available: <https://fastapi.tiangolo.com/> [Accessed: Sep. 23, 2025].
- [7] React, “React Documentation,” *ReactJS*. [Online]. Available: <https://react.dev/> [Accessed: Sep. 23, 2025].
- [8] Scikit-learn Developers, “Scikit-learn Machine Learning in Python,” *scikit-learn.org*. [Online]. Available: <https://scikit-learn.org/stable/> [Accessed: Sep. 23, 2025].